



nextwork.org

Create a Docker Container using Cursor



tahirgroot@gmail.com

<https://medium.com/@tahirbalarabe2/how-to-m...>



Introducing Today's Project!

I'm going to use natural language in Cursor to spin up a real PostgreSQL database container. No manual Docker commands. I'll connect the Docker MCP to Cursor, then ask Cursor to create and run the container for me. By the end, I'll have a live PostgreSQL container, and I'll even check its logs using only Cursor prompts. This is my first hands-on step into AI-driven containerization.

Key tools and concepts

The key tools I used include Cursor with AI chat, Docker Desktop, Docker MCP, Python and uv, PostgreSQL, Adminer, and Docker Compose. Key concepts I learned include containerization for isolating databases, MCP as a protocol letting AI control external tools like Docker, using natural language to create and manage infrastructure, Docker Compose for running multi container setups, and checking container logs for monitoring and debugging. I also learned how to visually access a database through a browser using Adminer.

Challenges and wins

This project took me approximately 60 to 90 minutes from start to finish. The most challenging part was ensuring the Docker MCP was correctly connected inside Cursor, especially installing Python and uv and getting the path set up. After that, creating the container and setting up Docker Compose felt smooth. Checking logs through natural language was surprisingly easy once everything worked. The initial setup required attention to detail, but the AI prompts made the rest very intuitive.

Why I did this project

I did this project today to learn how to control Docker using natural language inside Cursor through MCP. I wanted to move beyond editing files and actually create real infrastructure like databases just by chatting.



tahirgroot@gmail.com

NextWork Student

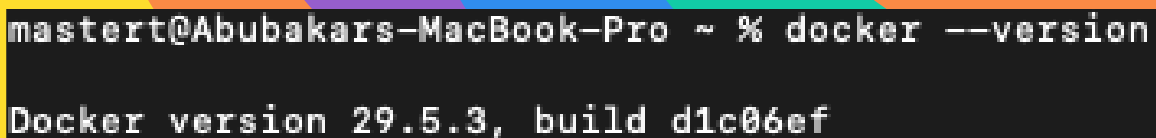
nextwork.org

Another MCP skill I want to learn is managing cloud resources, like spinning up AWS or Azure services through AI. I am also curious about MCP servers for Kubernetes, so I could deploy and scale containers conversationally. This project showed me that AI driven infrastructure is truly possible.



Setting Up Cursor and Docker Desktop

I'll start by installing Cursor, the AI code editor that lets me use natural language to control Docker. Then I'll install Docker Desktop, which will actually run my containers. These two tools work together: Cursor gives me the AI power, and Docker Desktop provides the engine. Once both are installed, I'll be ready to connect them and create my PostgreSQL container with simple chat prompts.

A terminal window with a black background and white text. The prompt is 'mastert@Abubakars-MacBook-Pro ~ %' and the command is 'docker --version'. The output is 'Docker version 29.5.3, build d1c06ef'. The terminal is overlaid on a colorful abstract background with wavy shapes in yellow, orange, purple, blue, and teal.

```
mastert@Abubakars-MacBook-Pro ~ % docker --version
Docker version 29.5.3, build d1c06ef
```

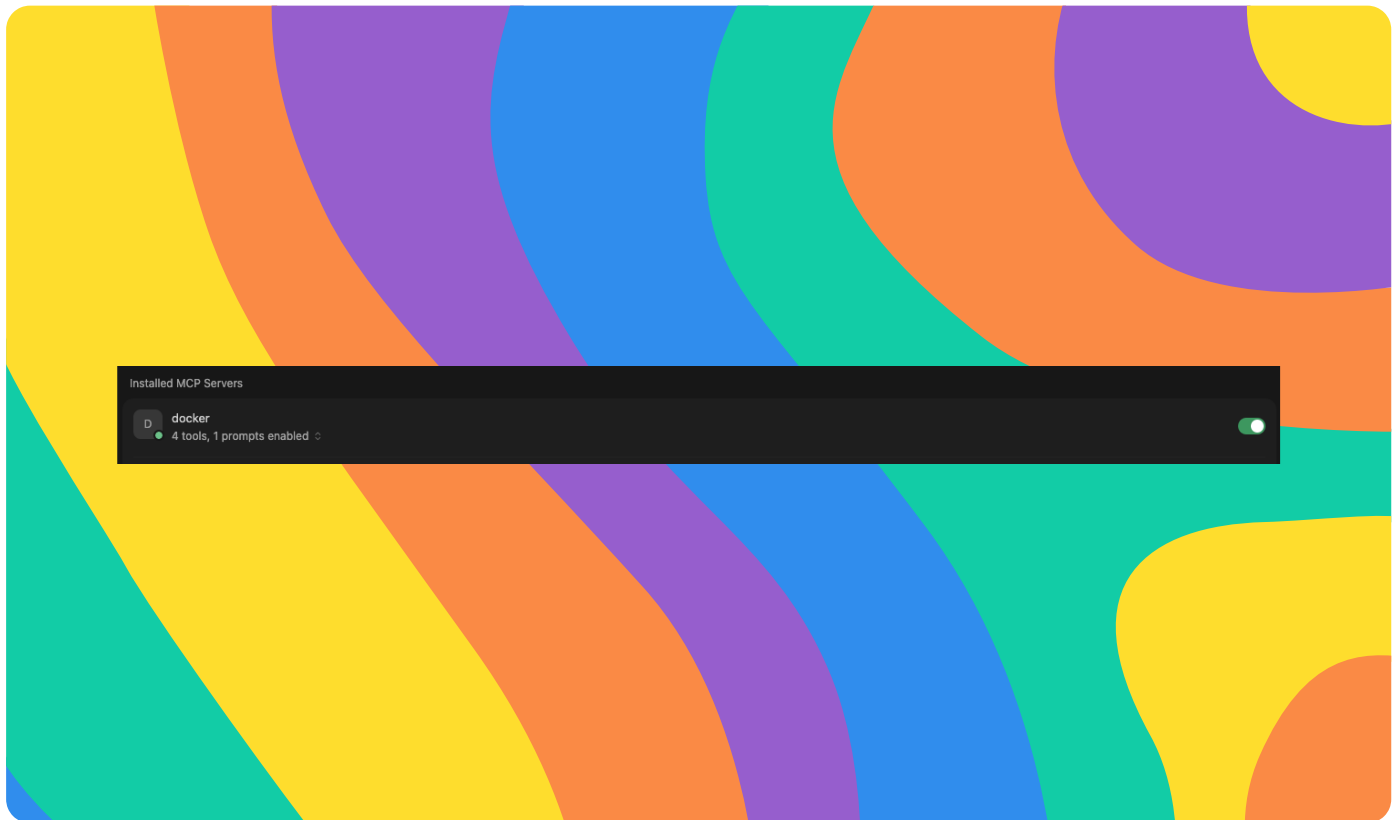
Why Docker containers?

I see why Docker containers are useful. They package an app like PostgreSQL with everything it needs to run, keeping my system clean and isolated. I don't install databases directly on my laptop anymore. Instead, I run them as lightweight, separate containers that I can start, stop, or delete easily. Docker Desktop gives me the engine and dashboard to manage all that. This keeps my development environment flexible and repeatable.



Connecting Cursor to Docker with MCP

In this step, I'm enabling the Docker MCP, which lets Cursor talk directly to Docker. Normally Cursor only edits files, but with MCP I give it real power to create and control containers using natural language. I'll install Python and uv, then connect the Docker MCP server inside Cursor. Once connected, I'll verify it so I know Cursor can now manage Docker for me.



Installing Python and uv

To set up the Docker MCP, I installed Python because many MCP servers, including this Docker one, are built in Python. I also installed uv, a fast package manager, to handle Python packages efficiently. Together they let me run the MCP server that connects Cursor to Docker. Without Python and uv, Cursor couldn't control containers. Now I'm ready to enable the Docker MCP inside Cursor.



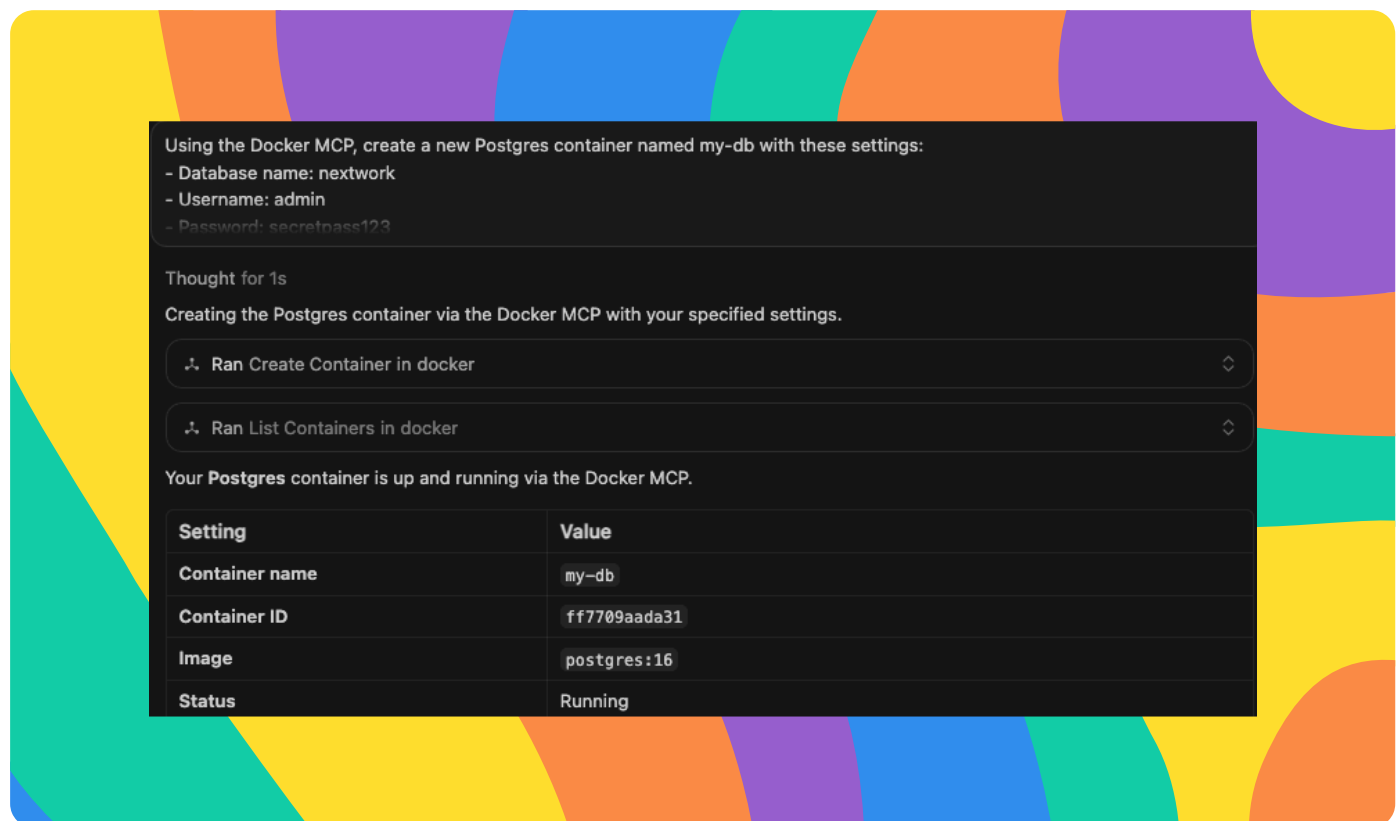
What the Docker MCP can do

The Docker MCP lets me control Docker directly from Cursor using natural language. The actions I can perform include listing all containers on my system, creating and starting a single container from an image, fetching logs from a container by name, deploying a full Docker Compose stack from YAML, and even describing a stack in plain English to generate and deploy it. I cannot stop, remove, or restart containers through this MCP, nor build images or manage volumes directly.



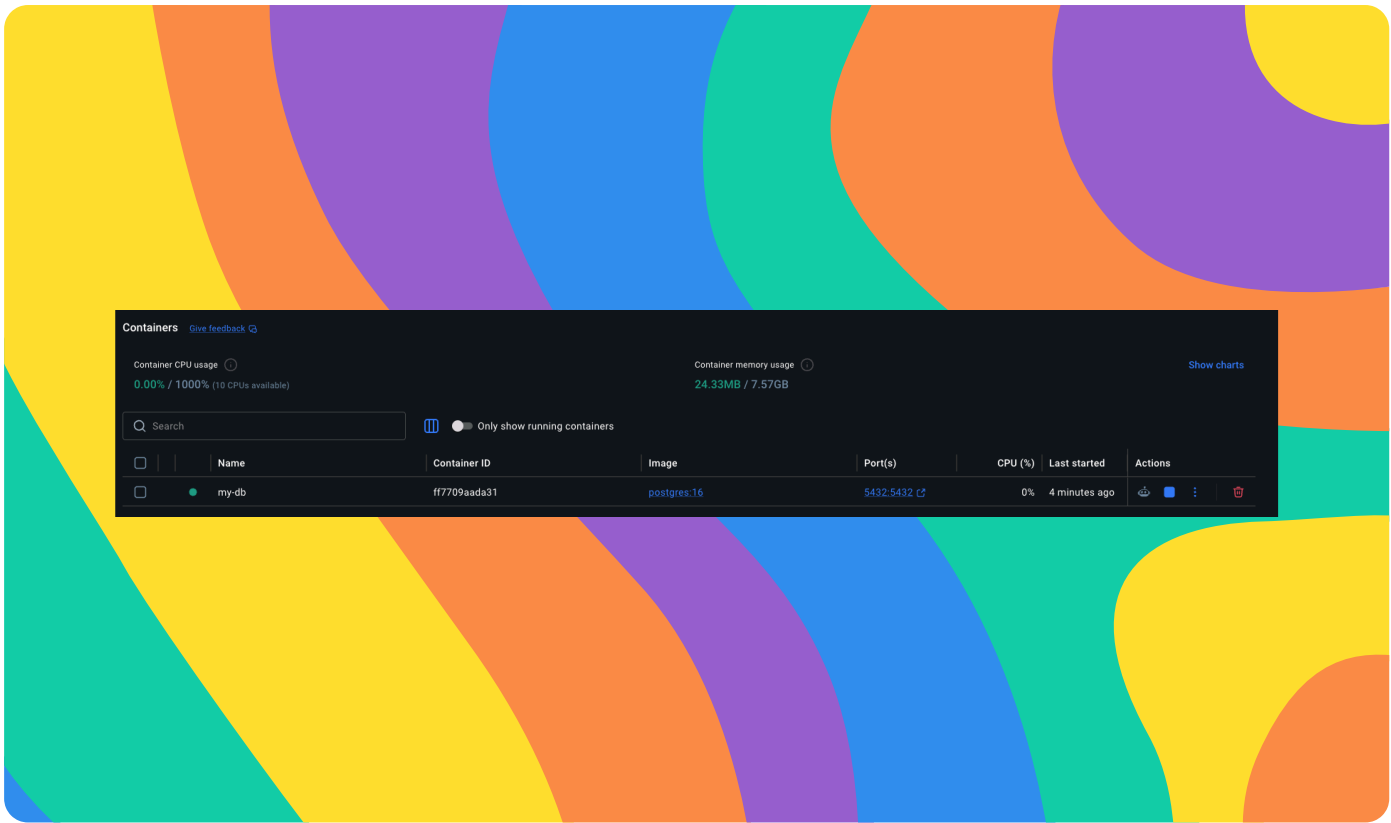
Creating My First PostgreSQL Container

In this step, I'm going to use the Docker MCP to ask Cursor to create a PostgreSQL container for me with a simple chat prompt. No manual Docker commands. I'll tell Cursor the image name, container name, port mapping, and environment variables like the database password. Then I'll check Docker Desktop to confirm my container is running. This shows how AI lets me manage infrastructure conversationally.



Verifying the container

opened Docker Desktop and clicked on Containers. Right there I see my-db with a green Running status. The container is alive. I didn't type a single docker run command. I just asked Cursor in plain English, and the Docker MCP did the rest. Seeing it running in the dashboard confirms that my AI driven setup actually works. Now I truly have a PostgreSQL database running inside a container, managed entirely through chat.



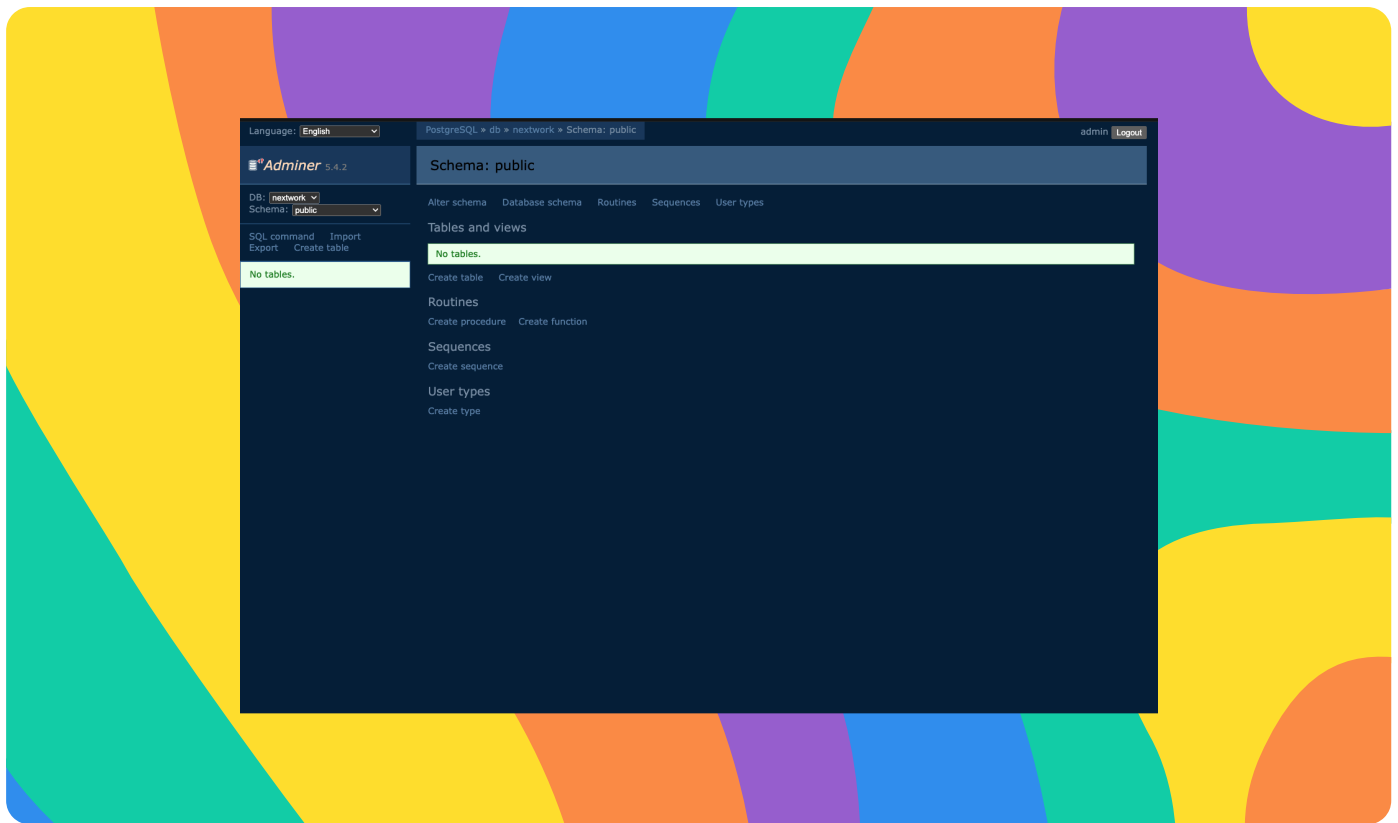


Orchestrating Multiple Containers with Docker Compose

In this step, I'm setting up a second container called Adminer, which is a web based database viewer. Docker Compose will help me define and run both PostgreSQL and Adminer together in one file. I'll create a project folder on my Desktop, write a docker-compose.yml file, then start both services. Once running, I'll open my browser to localhost:8081 and see my database visually for the first time. This makes inspecting data much easier than using the command line.

Setting up the docker-compose.yml file

I created a docker-compose.yml file that defines two services. The two containers running are PostgreSQL, named my-db, with database nextwork, username admin, password secretpass123, on port 5432. And Adminer, using the latest image, mapped to port 8081 on my host. This file lets me start both containers together instead of running separate commands. Now I can visually manage my database through Adminer while PostgreSQL runs in the background.



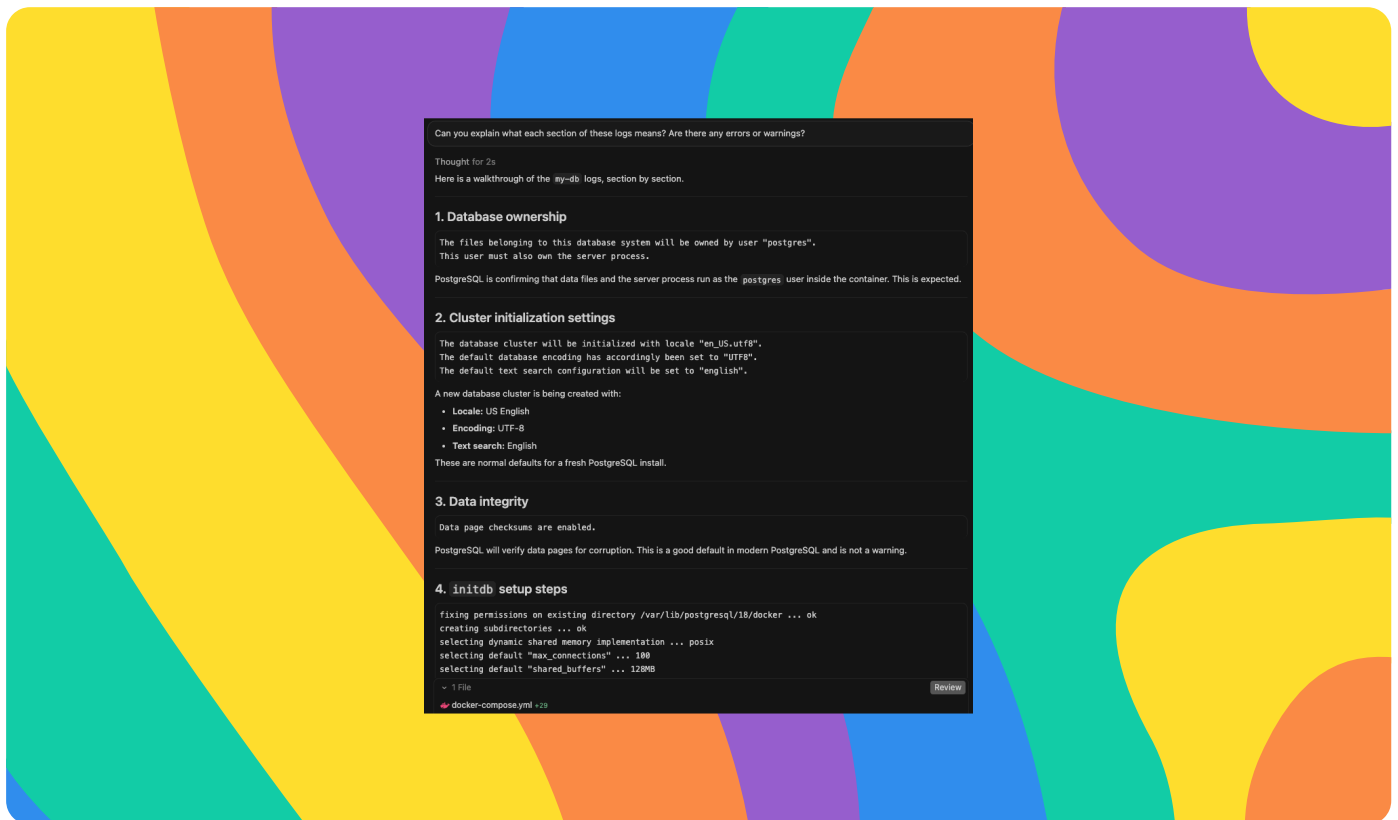
Accessing the database in the browser

I verified my database by logging into Adminer at localhost:8081. I entered the credentials from my docker-compose.yml file: server was my-db, username admin, password secretpass, and database nextwork. After clicking Login, I saw the nextwork database listed on the left side, completely empty. This confirmed that my PostgreSQL container is running correctly and that Adminer can communicate with it. Now I have a visual interface to manage my data instead of using only the command line.



Monitoring Container Logs

In this project extension, I'm going to use the Docker MCP inside Cursor to fetch live logs from my PostgreSQL container without opening a terminal. I'll ask Cursor something like "Show logs for my-db container." Then I'll read the output to see startup messages, connection attempts, and any errors. This helps me monitor my database health just like a real engineer. It proves that MCP gives me system level visibility through natural language alone.



What I learned from the logs

I checked my container logs without opening a terminal or Docker Desktop, by simply asking Cursor in natural language: "Using the Docker MCP, show me the logs for the my-db container." Cursor then used the MCP tool to fetch the logs directly inside the chat. I saw PostgreSQL's startup sequence, including version info, configuration settings, and connection readiness. Then I asked Cursor to explain each section, confirming there were no errors. This gave me full visibility into my container's health through plain conversation.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

